

面向对象编程

朱兮瑶



4.16

- Bug:**
- 1. python 中符号用英文模式.
 - 2. `print( )` 顶格写
 - 3. 两个 `print` 不能写在同一行 (`SyntaxError: 语法错误`)

变量名 (variable) 赋值 值 (value)

- 1. "只能由数字、字母、_ (下划线) 组成.
- 2. 不能以数字开头
- 3. 不能是关键字. (`"print"`, `"for"`, `"in"` etc.)
- 4. 严格区分大小写.

哈尔滨工业大学

薪火笔记社

True or False

`type( )` `str` (字符串), `int` (整型), `float` (浮点型), `bool` (布尔型)

`xtext = "She said "OK""` #号不能滥用

`vtext = "She said \"OK\""` (\ 取消特殊含义)

`long_text = "..."` $\Rightarrow$ `.....` (没有空格)

`string1 = "..."`
`string2 = "..."`

`long_text = """..."""` $\Rightarrow$ `'''...'''` 而 `"""..."""` 是注释.

`string3 = string1 + string2`

`text = "Python BootCamp"`

"P": `text[-1]` | `text[len(text)-1]`

`text[ ]` `[1:6]` 取 "1~5", `[ :6]` 取 "0~5", `[7: ]` 取 "7~最后"

`xtext[6] = "_"` (`'str' object doesn't support item assignment`)

method, method 不会改变 Value 值

"Python Boot Camp".lower() 小写

.upper() 大写

(空格换行符, 制表符(\t))

.strip() 移除 字符串两端的空白字符,

.count(Python) 统计 字符出现的次数.

.find() 返回 字符串首次出现的索引 (从 0 开始)
(未找到则返回 -1)

.startswith('P') $\Rightarrow$ True 检查 前缀开头

算术运算符.

+ - * / 使用算术运算符, 商一定是浮点数, 且除数不能为 0.

// 取整除, 取商的整数部分向下取整 (只有有小数, 就忽略小数)

% 取余数, 只取余数部分.

** 幂, $m ** n$: m 的 n 次方 (m^n)

(使用算术运算符, 其中若有浮点数, 结果也会用浮点数表示)

优先级排序: 幂 (最高优先级) > 乘, 除, 取余, 取整除 > 加, 减

$b = b - 1$ / $b -= 1$ 赋值运算符必须连着写, 中间不能有空格.

"2" + "2" = "22" ! 不等于

(...) or (...) / (...) and (...) / not (...)

input() 输入函数,

input(prompt) prompt 是提示, 会在控制台显示
输出的结果是 "str"

name = "Doctor Strange"

heads = "2" + 格式化 f"{'表达式'}"

arms = "3"

print() $\Rightarrow$ Doctor Strange has 2 heads and 3 arms.

1. name + "has" + heads + "heads and" + arms + "arms."
2. f"name" has {heads} heads and {arms} arms.
3. "{ }" has { } heads and { } arms. format(name, heads, arms)
4. "%s" has %s heads and %s arms. %(name, heads, arms)

格式化输出

%? :
占位符 (生成一定格式的字符串) print(我是 %? 的 %? %?) $\Rightarrow$ 我是 %? 的 %?

%s 字符串 (常用)

占位符 只是占位置, 并不会被输出

%d 整数 (常用)

数字 设置位数, 不足前面补零 (0)

%f 浮点数 (常用)

默认 后几位小数 遵循四舍五入原则

%.4f 浮点数 (常用) 设置小数的位数

name = "bing bing"

print("我的名字: %s" % name)

a = 123 输出123 若超出则原样.

print("%04d" % a) $\Rightarrow$ 000123

a = 1.2345673

print("%.4f" % a) $\Rightarrow$ 1.2346

print("%.4f" % a) $\Rightarrow$ 1.2345

4.18

• if 判断函数

if 要判断的条件 (condition):
缩进 条件成立的时候要做的事情。

• if-else: (二选一)

if 条件:
____ 满足条件时要做的事情

else:
____ 不满足条件时要做的事情
(else 后面不需要添加任何条件)。

• if-elif: (多选一)

if 条件 1:
____ 满足条件 1 要做的事情

elif 条件 2:
____ 满足条件 2 要做的事情

else:
____ 不满足条件 1, 2 要做的事情

• if 嵌套

if 条件 1:
____ 事情 1
 if 条件 2:
 ____ 事情 2

else:
____ 不满足条件 1 做的事情

哈尔滨工业大学
薪火笔记社

} condition 1 和 condition 2
顺序不能改

自定义函数:

```
def multiply(x, y):  
    product = x * y  
    return product
```

print 将内容输出到控制台
return 将内容返回并返回一个值
并同返回值可以通用
需 print() 函数调用
若函数无 return 为 None
Function signature
Function body
Return statement
调用函数 (保证在函数内)

multiply(1, 2)
实参

x=2
x=3
x => 3
一般函数中
x 值会被替代

x=5 (global scope)
def func():
 y = x + 5
 print("Inside 'func', y has the value", y) => 10
func()
print("Outside 'func', x has the value", x) => 5
print("Outside 'func', y has the value", y) X Error
global y
=> 设为全局变量
外函数 x 值 (global scope) 不会被内函数替代。
nonlocal x 改变上一层值

global y
=> 设为全局变量
外函数 x 值 (global scope) 不会被内函数替代。

Outside function is visible inside.
Inside function is invisible outside.

(变量)
必需参数
默认参数
可变参数
关键字参数

def func(a, b): ... func(a, b) 位置参数
def func(a=1): ... func() 位置参数必须在此之前
def func(*args): 传入的数量可变, 以元组形式接收
def func(**kwargs): 以字典形式接收 func(name='bj')

参数一致
位置参数
位置参数必须在此之前
传入的数量可变, 以元组形式接收
以字典形式接收

range, String, List, Tuple

For 循环

for 临时变量 in 可迭代对象: step.
range(, , ,)

for i in range(1, 11): (用来记录循环次数) 1 取不到
print(i) ⇒ 1, 2, 3 ..., 10. 循环目前不向后

continue

for i in range(1, 11):

if i == 5:

continue

print("We are here")

print(i) ⇒ 1, 2, 3, 4, 6, ..., 10.

只能放在循环中使用

退出本次循环, 下一次循环继续执行

break

....
....

break

.... ⇒ 1, 2, 3, 4.

某条件满足时, 退出循环

pass

....
....

pass 不产生任何影响

.... ⇒ 1, 2, 3, 4, We are here, 6, ..., 10

While 循环

while 条件:

—— 循环体 (条件满足时要做的事情)

—— 改变变量. (如果没有改变变量, 就会一直循环)
(死循环)

while True: / False: (不会执行)

—— 循环体 (要循环做的事情)

counter = 1 定义一个变量.

while counter < 11:

print(counter)

counter = counter + 1 改变变量

Nested 嵌套循环

for row in range(3):

for column in range(3):

number = row * 3 + column + 1

print(" ", number, end=" ")

print("\n") 换行符

用空格/其他分隔符代替换行.

↓

1 2 3

4 5 6

7 8 9

转义字符

`\t` 制表符 (通常表示空四个字符, 也称缩进)

`print("Sixs\tar")` $\Rightarrow$ Sixs ar

`\n` 换行符 (表示将当前位置移到下一行开头)

`print("哈哈\n嘻嘻")` $\Rightarrow$ 哈哈
嘻嘻

`\r` 回车 (表示将当前位置移到本行开头)

`print("Sixsta\r sdhdfh")` $\Rightarrow$ sdhdfh
前面的将消失

`\\` 反斜杠符 (两个斜杠表示一个斜杠)

`print("Sixs\\tar")` $\Rightarrow$ Sixs\tar

`print("Sixs\\tar")` $\Rightarrow$ Sixs\tar

4.22

质数函数:
(is_prime())
(质数大于1的自然数)

```
def is_prime(n):  
    if n <= 1:  
        return False  
    for i in range(2, int(n ** 0.5) + 1):  
        if n % i == 0:  
            return False  
    return True
```

或: def is_prime(n):

1° 0, 1 if n <= 1:
return False 0, 1 不是质数.

2° 偶数 if n == 2:
return True 2 是唯一一个偶数中的质数.

if n % 2 == 0:
return False (even)
其他偶数不是质数

3° 奇数. for i in range(3, n):
if n % i == 0:
return False 奇数.

return True

print("hello", "there") ⇒ hello there

print("hello", "there", sep=" ", end="!") ⇒ hellothere!

下标/索引 0 1 2 ...
... 2 -1

Date Structure

String: (有顺序, 不可以改变)

(字符串) string = " " 只能用str类型.

string1 = "Hello", string2 = "Python"

string1 + string2 $\Rightarrow$ "HelloPython", string1 * 2 $\Rightarrow$ "HelloHello"

string1[1] = e, string[1:4] = ell $\leftarrow$ 切片

print("H" in string1) $\Rightarrow$ True print("h" not in string1)

P3: find: 检测某个字符串是否包含在字符串中并返回下标
(无 $\rightarrow$ -1) // find(字符串, 开始下标, 结束下标) 包前不包后.

index: (无 $\rightarrow$ 报错)

count: 返回某个字符串在某个字符串中出现的次数, 没有则返回0

startswith: 是否以某个字符串开头.

end with: 是否以某个字符串结尾.

isupper: 是否所有的字母都是大写 islower

capitalize: 第一个字符大写

replax: 替换

replax(旧内容, 新内容, 替换次数)

split: 指定分隔符来切字符串 * 以列表形式返回.

st = "hello, python"

print(st.split(',')) $\Rightarrow$ ["hello", "python"]

若不包含分隔内容, 则以整体返回. (',', 次数)

join/合成 " ".join() 分隔符.

Tuple: (有顺序, 不可改变)

(元组)

```
tuple = ('John', 'Red')
```

```
tuple[1] = 'John' [ : ]
```

```
X tuple[1] = 'David'
```

are immutable (不能改变值)

区别1°

只有1个时, 以","结尾.

可以由不同类型的元素组成

(in)

```
if "Red" in tuple:
```

```
    print("There is red")
```

```
else:
```

```
    print("There is no red") ⇒ There is red.
```

区别2°

只支持查询操作, 不支持增删改操作

Count, index, len 用法一样

格式化输出后面的()本质上就是一个元组.

```
name = "bingbing"
```

```
age = 18
```

```
print("%S的年龄是:%d" % (name, age))
```

```
或 info = (name, age)
```

```
print("%S的年龄是:%d" % info)
```

元组.

列表嵌套

List: (有顺序, 可以改变)

(列表) `list = [1, "John", "Red"]` 可以由不同类型的元素组成

1° `list[1] = "John"` `[:]` 若 `li = [1, 2, 3, 4]`, `li[1][0] = 3`

修改元素 `list[1] = "David"` mutable (可以改变)

2° 添加元素: `li = ['one', 'two', 'there']` 整体添加

`append` `li.append("four")` $\Rightarrow$ `['one', 'two', 'there', 'four']`

`extend` `li.extend("four")` $\Rightarrow$ `['one', 'two', 'there', 'f', 'o', 'u', 'r']`

一定要用可迭代对象 $\leftarrow$ 分散添加 将另一个类型中的元素逐一添加

`insert` `li.insert(0, '0')` $\Rightarrow$ `['0', 'one', 'two', 'there', 'four']`

指定位置插入元素, 若有, 则原有元素后移

`li.extend(4)` X

3° 查找元素:

`in/not in`, `index`, `count`

`del list`

4° 删除元素:

`del` `del list[2]` 根据下标删除, 报错, 不存在

`pop` `li.pop()` 默认删除最后一个元素 $\leftarrow$ 不能超出

不能指定元素删除, 只能根据下标删除, 范围

`remove` `li.remove("John")`

根据元素删除, 若不存在则报错

默认删除第一个找到的元素

5° 排序:

`li = [1, 5, 3, 2, 4]`

`sum(li) = 15`

`li.sort()` 从小到大

$\Rightarrow$ `sorted(li, reverse=True)`

`li.reverse()` 倒序

6° copy: `li1 = li` 则 `li1` 和 `li` 一起改变, `li1 = li[:]` 只有 `li1` 改变

List Comprehension (列表推导式)

- 1° [表达式 for 变量 in 列表] range(), 可迭代对象
li = [1, 2, 3, 4] [print(i*5) for i in li] $\Rightarrow$ [5, 10, 15, 20]
- 2° [表达式 for 变量 in 列表 if 条件]
li = [] [li.append(i) for i in range(1, 11) if i%2==1]
 $\Rightarrow$ [1, 3, 5, 7, 9]

3° numbers = (1, 2, 3, 4, 5)

square = []

for num in numbers:

square.append(num**2) $\Rightarrow$ [1, 4, 9, 16, 25]

哈尔滨工业大学
薪火笔记社

Dictionary: (无顺序, 可以改变) key value
(字典) ~ Dictionary = { "France": "Paris",
"Spain": "Madrid",
"Italy": "Rome" }

字典中键具备唯一性, 但值可以重复。
重复不会报错, 键名重复的后面会被后面覆盖。

1° 查看元素: Dictionary["France"] key ⇒ "Paris"
查看元素不可以根据下标, 根据键名。
键名不存在时报错。

get Dictionary.get("France") key ⇒ "Paris"
键名不存在时, 返回 None
(默认值) 若不存在时, 返回自己设置默认值

2° 修改元素: Dictionary["Spain"] = "..."
键名存在就修改, 不存在就新增

3° 删除元素 del del Dictionary
del Dictionary["Spain"] 删除键值对。
若无比键名, 则报错。

clear (清空整个字典里面的东西, 但属于这个字典)
Dictionary.clear() ⇒ {}

pop Dictionary.pop(" ") 删除指定键值对。
若无比键名/没有, 则报错。
Dictionary.popitem() 默认删除最后一个

len: `len(Dictionary)` $\Rightarrow$ 3 (键值对的对数)

keys: 返回字典中包含的所有键值

`Dictionary.keys()` $\Rightarrow$ `Dictionary_keys(['', ''])`
`for i in Dictionary.keys():` 只取出键值。

values: 返回字典中包含的所有值

`Dictionary.values()` $\Rightarrow$ `Dictionary_values(['', ''])`
`for i in Dictionary.values():` 只取出值。

items: 返回字典中包含的键值对。(以元组形式)

`Dictionary.items()` $\Rightarrow$ `Dictionary_items([''], ['']), [''], [''])`
`for i in Dictionary.items():` $\Rightarrow$ `(' ', '')`

`.isalpha()`: 是否仅由字母组成

`.isdigit()`: 是否全部为数字

`.isalnum()`: 是否由字母或数字组成

`isinstance(value, (int, float))` (int, float) 检查 value 值是否为 int 或 float

Set: (无顺序, 不可以修改) 里面的元素是唯一的 (自动去重)
(集合) 可添加, 删除.

Set = {"apple", "banana", "cherry"} 可以由不同变量
再集合: Set()

字母顺序不一样, 数字一样.
因 hash 表 (了解)

1° 添加元素: add set.add() 添加的是一个整体
如果元素已存在, 则不进行任何操作. 只能添加一个元素.
update set.update() 可添加多个元素, 集合添加.

2° 删除元素: remove set.remove() 若无此元素, 则报错.
pop set.pop() 对集合进行了无序排列, 左边第一个删除.
discard set.discard() 不报错
指定删除, 若没有则不会发生任何改变.

4.24.

计算文中出现最多次数的单词 (Code)

1° words = text.lower().split() $\Rightarrow$ Convert to lowercase and split into words (List)

cleaned_words = []

for word in words:

cleaned_words.append(word.strip(',!?"()')) $\Rightarrow$ 去除符号.

word_freq = {}

for word in cleaned_words:

if word in word_freq:

word_freq[word] += 1

else:

word_freq[word] = 1 $\Rightarrow$ 创建字典 {word, count}

most_common_word = ""

max_count = 0

for word, count in word_freq.items():

if count > max_count:

most_common_word = word

max_count = count

$\Rightarrow$ 取出出现次数最多的

print(f"The most frequent word is '{most_common_word}'
which appears {max_count} times")

2° from collections import Counter
words = article.lower().split() $\Rightarrow$ Convert to lowercase and split into words
words = [word.strip('.,!";?()') for word in words] $\Rightarrow$ remove punctuation from words
word_counts = Counter(words) $\Rightarrow$ Count word frequencies $\Rightarrow$ 取出每个 List, List 结果 index
most_common = word_counts.most_common()[0]
 $\Rightarrow$ Find the most common word
print(f"The most frequent word is '{most_common[0]}'
which appears {most_common[1]} times")
longest_word = "" $\Rightarrow$ Initialize variables
max_length = 0
for word in cleaned_words:
 if len(word) > max_length:
 longest_word = word
 max_length = len(word)
print(f"The longest word is '{longest_word}' with length
{max_length}")

匿名函数: 函数名 = lambda 形参: 返回值(表达式)
普通函数 def add(a,b):
return a+b
print(add(1,2))
匿名函数 add = lambda a,b: a+b
print(add(1,2))

参数形式:

1° 无参数 funa = lambda : "..." print(funa()) => "..."
2° 有参数 funb = lambda name: name (加外带参数)
3° 默认参数 func = lambda name, age=18: (name, age)
返回值以元组形式

4° 可变参数 funb = lambda *args: args 元组
5° 关键字参数 func = lambda **kwargs: kwargs 字典

三目运算 真结果 if 条件 else 假结果

a=8

b=5

lambda 语句
if 判断
print("a比b小") if a<b else print("a大于等于b")
func = lambda a,b: "a比b小" if a<b else "a大于等于b"
print(func(8,5))
lambda 只能实现简单的逻辑。

内置函数:

查看: `import builtins`

大写字母开头一般是内置常量名,

`print(dir(builtins))` $\Rightarrow$ 小写字母开头一般是内置函数名.

`set()`, `list()`, `tuple()`

`abs()`: 返回绝对值

`print(abs(-10))` $\Rightarrow 10$

`(,)` `[]` `{}`
(tuple, list, set)

`sum()`: 求和

`print(sum(1, 2, 3))` $\Rightarrow 6$

可迭代对象 (iterable)

`min()`: 求最小值

`print(min(4, 1, 8))` $\Rightarrow 1$

`print(min(-8.5, 5, 10))` $\Rightarrow -8.5$

`max()`: 求最大值

`print(max(4, 1, 8))` $\Rightarrow 8$

`max` $\Rightarrow 8$

`zip()`: 将可迭代对象作为参数, 将对象中对应元素打包成一个元组.

`Li1 = [1, 2, 3]`

`Li2 = ['a', 'b', 'c']`

可迭代对象

`print(zip(Li1, Li2))` $\Rightarrow \langle zip \text{ object at } \dots \rangle$

1° `for i in zip(Li1, Li2):` $\Rightarrow (1, 'a') (2, 'b') (3, 'c')$ 元组

读取: 当个数不一样时, 按长度最短的组合

2° `print(list(zip(Li1, Li2)))` $\Rightarrow [(1, 'a'), (2, 'b'), (3, 'c')]$

`map()`: 可以对可迭代对象中的每一个元素进行映射, 分别去执行

`map(func, iter())` 自定义函数, 可迭代对象. (只需要函数名)

`mp = map(func, Li1)`

`reduce()`: 先把对象中的前两个元素取出, 计算出一个值然后保存着.

接下来把这个计算值跟第三个元素进行计算

`from functools import reduce`

(可迭代对象)

`reduce(function, sequence)` 有两个参数的函数, 序列

`res = reduce`
`print(res)`

模块 (module)

一个py文件就是一个模块,即导入一个模块本质上就是执行一个py文件

1° 内置模块: random, time, os, logging, 直接导入即可使用

Import _

2° 第三方模块: (第三方库):

cmd窗口输入 pip install 模块名

3° 自定义模块: (不要与内置模块产生冲突)

导入模块: 1° import 模块名1, 模块名2

调用: 模块名.功能名

2° from 模块名 import 功能1, 功能2 只需函数名.
从模块中导入指定的部分

调用: 功能名() 没有导入需要的功能名则报错.

3° from 模块名 import * 导入所有功能

4° import 模块名 as 别名 给模块起别名

调用: 别名.功能名

5° from 模块名 import 功能名 as 别名

调用: 别名()

如: 在当前程序执行(即自己执行自己)

内置全局变量: if __name__ == "__main__":

__name__ 用来抑制py文件在不同的运行场景执行不同的逻辑

文件被当作模块导入 if __name__ == 模块名

包 (package):

项目中结构中的文件夹/目录。

与普通文件夹区别: 包是含有 `__init__.py` 的文件夹
将有联系的模块放在同一个文件夹下, 并且创建一个名为
`__init__.py` 的文件夹, 有效避免模块名称冲突问题

1° Import 包, 首先执行 `__init__.py` 中代码

2° from 包 Import 模块
模块、功能名()

`__all__`: 本质上是一个列表, 可以控制要引入的东西
`__all__ = ['模块名']` 相当于导入 `__init__.py` 中定义的模块
可以导入同一包中其他模块

哈尔滨工业大学
新火笔记社

Standard Library:

Copy (module)

```
animals = ["lion", "tiger", "cheetah"]
```

```
large_cats = animals
```

```
large_cats.append("Leopard")
```

```
large_cats ⇒ [..., "Leopard"]
```

```
animals ⇒ [..., "Leopard"]
```

animals[0:]

Import copy / 不会改变原来列表

```
large_cats = copy.copy(animals)
```

```
large_cats.append("Leopard")
```

```
large_cats ⇒ [..., "Leopard"]
```

```
animals ⇒ [...]
```

math (package)

mathematical constants: π , e ,

mathematical functions: $\log$, $\cos$

utilities: ceil , floor

```
import math
```

```
print (math.log(5.2)) ⇒  $\log_2 5.2$ 
```

```
math.pi ⇒  $\pi$ 
```

```
math.floor(2.6) ⇒ 2
```

```
math.ceil(2.6) ⇒ 3
```

```
math.round(2.6) ⇒ 3
```

```
math.sin( )
```

```
math.cos( )
```

整数
向下取整, 返回值是不大于()的最大整数

向上取整, 返回不小于()的最小整数

四舍五入取整

time (package) deal with time and timing-related things

`import time`

`print(time.time())` $\Rightarrow 1745630446.69053$ January 1st 1970 00:00:00
the seconds that have passed since r

`time.sleep(2)` the script will now pause for 2 seconds

求函数运行所需时间: `def func(): ...`

`import time`

`start_time = time.time()`

`func()`

`print("{:.4f} second".format(time.time() - start_time))`

取4位小数
新火笔记社

time object

`now = time.localtime()`

`print(now)` $\Rightarrow$ `time.struct_time(tm_year=2025,`
`tm_mon=4, tm_mday=26, tm_hour=9,`
`tm_min=21, tm_sec=19, tm_wday=5,`
`tm_yday=116, tm_isdst=0)`

`print(now.tm_year)` $\Rightarrow 2025$

datetime: `import datetime`

`datetime.datetime.today().year` $\Rightarrow 2025$

$\Rightarrow$ `datetime.datetime`
`(2025, 5, 8, 15, 25, 2, 7203)`

External Library:

Install Third-Party Packages with pip

pip --version conda active

pip install --upgrade pip upgrade pip

pip list list all of the packages you have installed

pip install numpy install the numpy package

pip install numpy==1.16.5 install specific version

pip uninstall numpy uninstall a package

random 模块 (import random) axis=0, 沿行方向, 即垂直

用于实现各种分布的伪随机数生成器,

可以根据不同的实数分布来随机生成值

random.random() 产生大于0且小于1之间的小数

random.uniform(1, 3) $\Rightarrow 2.93 \dots$ 产生指定范围的随机小数

random.randint(1, 3) $\Rightarrow 1/2/3$ 产生指定范围内的随机整数,

包括开头和结尾

random.randrange(start, stop, step) 包含开头但是不包含结尾

指定产生随机的步长, 随机选择一个数据

(2, 4, 2) $\Rightarrow 2$ create() integers

measurements = [random.randint(1, 200) for _ in

range(1000000)] $\Rightarrow [181, 166, 179, 163, 179]$

list_time = %timeit -o sum(measurements) /

所需时间检查 len(measurements) $\Rightarrow 2.1 \text{ ms} \pm 9.1 \mu\text{s}$ per loop
objects actually support addition

numpy: 数据统计, 随机数生成 (提高运算速度)

import numpy as np

data type: ndarray (stands for n-dimensional array)
数据类型: measurements_array = np.array(measurements)

numpy-time = %timeit -o np.mean(...array)
数组元素求平均值

np.arange(5) ⇒ array([0, 1, 2, 3, 4])

start = , stop = , step = , dtype = float

array: 创建数组

一维数组 np.array(measurements) ⇒ array([181, 166, ...])

object, dtype=None, copy=True, order=None

C 为行, F 为列, A 为 Fortran 顺序 subok=False, ndmin=0

dtype = np.int / np.float

最小维数, shape

等差数列

np.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None) ⇒ int / float

是否包含 stop

zeros: 创建数组

np.zeros(shape, dtype=float, order='C')

一维: np.zeros(2) ⇒ array([0., 0.])

二维: np.zeros((1, 2)) ⇒ array([[0., 0.]])

三维: np.zeros((1, 2, 3)) ⇒ array([[[0., 0., 0.], [0., 0., 0.]]])

指定形状
zeros_like. 同 random 确定随机种子, 后每次不重
np.random.seed(10)
np.random.randint(1, 10, 10)

a = np.arange(4) ⇒ array([0, 1, 2, 3])

np.zeros_like(a) ⇒ array([0, 0, 0, 0])

aa = np.random.random(size=(1, 2))

aa.shape ⇒ (1, 2)

np.zeros_like(aa) ⇒ array([[0., 0.]])

ones 创建:

np.ones(shape, dtype=None, order='C') 同 zeros

— .dtype ⇒ return data type of the array

(booleans, ints, unsigned ints, floats or complex numbers)

— .size ¹⁰ of various byte sizes (strings or python objects)

— .shape ⇒ a tuple with the number of elements in each

— .ndim ⇒ the number of dimensions in total. dimension

索引和切片: 从0开始计数 (不能超) 负: 从-1开始

— .reshape(shape) 改变数组维数, 元素个数需保持一致

当不知道元素个数时: 3维 → 1维 — .reshape(-1)

变一维: a.reshape(-1)
a.ravel()
a.flatten() 或 a.shape = (6, 4) shape 属性
a.resize(shape)

数学函数

`np.abs()`、`np.fabs()` 计算整数、浮点数的绝对值

`np.sqrt()` 计算各元素的平方根

`np.reciprocal()` 计算各元素的倒数

`np.square()` 计算各元素的平方

`np.exp()` 计算各元素的指数 e^x $10/2$ 的对数

`np.log()`、`np.log10()`、`np.log2()` 计算各元素的自然对数, 底数为 e

`np.sign()` 计算各元素的符号, 1 (整数), 0 (零), -1 (负数) 四舍五入

`np.ceil()`、`np.floor()`、`np rint` 对各元素分别向上取整、向下取整

`np.around()` $\rightarrow$ `decimals = 0` (默认) 四舍五入 1.2 保留几位小数
负数 整数部分四舍五入到小数点后一位

`np.modf()` 将各元素的小数部分和整数部分以两个独立的数组返回

`np.cos()`、`np.sin()`、`np.tan()` 将各元素求对应的三角函数

`np.add()`、`np.subtract()`、`np.multiply()`、`np.divide()` $x \div y$

`arr @ arr = np.sum(arr * arr)`

数组的拼接

列表中: `— extend()`

数组中: `np.concatenate((a1, a2...), axis)`

= 三维拼接时 shape 都一样

二维数组: `hstack()` 水平堆叠 `np.hstack((a, b))`

`vstack()` 垂直堆叠 `np.vstack((a, b))`

三维数组: `+ dstack()` `np.dstack((a, b)) = axis = 2`

更高阶 $\text{np.append}(a, b) \Rightarrow [a, b]$ Array-likes
 $\text{np.stack}(a, b) \Rightarrow \begin{bmatrix} [a], \\ [b] \end{bmatrix}$

Mask: $\text{arr} = \text{np.arange}(1, 6) \Rightarrow \text{array}[1, 2, 3, 4, 5]$
 $\text{mask} = \text{np.array}([True, False, True, False, True])$
 $\text{arr}[\text{mask}]$ bool 只输出 True 对应的值 对每个元素都进行判断
 $\text{arr} < 5 \Rightarrow$ 输出 bool 结果 $\text{array}[True, True, ..., False]$
 $\text{arr}[\text{arr} < 3] \Rightarrow \text{array}([1, 2])$

哈尔滨工业大学
薪火笔记社

5.8

`enumerate()`: 返回索引和对应的元素值

```
numbers = [20, 13, 17]
for i, v in enumerate(numbers):
    print(i, v)
```

⇒

index	value
0	20
1	13
2	17

Try: `num = input()` ← bob

`num = int(num) * 1.9`

`print(num)`

x Value Error

Handle →

try: if no error happens:

`num = input()` ← bob

`num = int(num) * 1.9`

`print(num)`

except: if error happens:

`print("Please input a number")`

bob ⇒ please input a number

哈尔滨工业大学
薪火笔记社

Object Oriented Programming:

object

Class:

`class Rectangle:`

`def __init__(self, w, h):` dunder methods

`self.width = w`

`self.height = h`

`def area(self):` function

`return self.width * self.height`

`def __str__(self):` define what print means for objects of this class

`return f"width = {self.width}; height = {self.height}"`

`rec1 = Rectangle(6,4)` `rec1` is an object of class `Rectangle`

⌞ `rec1 = Rectangle()`

`rec1.__init__(w=6, h=4)`

`print(rec1.area())` `rec1.area()` is an function of the `Rectangle`

`type(rec1)` $\Rightarrow$ `__main__.Rectangle`

当无 `def __str__(self):` 时

$\hookrightarrow$ `print(rec1)` $\Rightarrow$ `<__main__.Rectangle object at 0x1060a812>`

有 `def __str__(self):` 时

`print(rec1)` $\Rightarrow$ `width: 6 ; height: 4`

`is_palindrome()`:

solution 1: 1. reverse the word

2. compare the original with the reverse

solution 2: use the index one from the begin one from end

and compare each pair of chars

reverse: `word_L = list(word)` 不会改变 `word_L` `word_L = list(word)`

1° `word_r = copy.copy(word_L)` 2° `word_r = word_L[::-1]`

`word_r.reverse()`

`word_L == word_r`

`word_L == word_r`

5.9

Read / Write text files `"/data/f.txt"`

`myfile = open( file="f.txt", mode='w' )`
 ↓
 an object of class file
 file = location and name of the file
 'r': read ; 'w' = write ; 'a' = append (add content to an existing file)
`myfile.write("hi")`
`myfile.write("\n")` "`\n`" means a new line
`myfile.close()` a function to say that you finish working with the file

`myfile = open( file="f.txt", mode="r" )`
`text = myfile.read()` read the entire content of the file as a single string
`lines = myfile.readlines()` read the entire content of the files as a list of lines
`myfile.close()`
`print( lines )` $\Rightarrow$ `['hi\n']`
`lstrip(" ").rstrip("\n")` 从() 删除

Visualization in Python

`import matplotlib.pyplot as plt`
 Data: `x=[1,2,3]`
`y=[10,15,40]`
 图例创建图形 坐标轴对象
`fig, ax = plt.subplots( rows=1, cols=1, figsize=(2,2) )`
`ax.plot(x,y) / ax.scatter(x,y)`
`plt.show()`
 plot: 直线
 scatter: 点

+ marker="1" / "o" / "x", label=""
 + ax.legend(bbox_to_anchor=(1.02, 1))
 添加标签 图例

ax.plot(x, y, linestyle=":", linewidth=2, color="green")
 虚线

ax.scatter(x, y, color="red", s=50)
 散点图

ax.bar(x, y, color="skyblue", edgecolor="navy", linewidth=1.5, width=0.8)
 柱状图

ax.axhline(y=avg, color="red", linestyle=":")
 水平线 由 avg 指定

ax.set_title()
 设置标题

ax.set_xlabel()
 设置 x 轴标签

ax.set_ylabel()
 设置 y 轴标签

ax.text(3, avg, "average", c="red")
 文本内容

二维折线图:

哈尔滨工业大学
 薪火笔记社

x = np.linspace(0, 10, 100)

y = np.sin(x)

plt.plot(x, y, color="blue", label='sin(x)')

plt.xlabel()

plt.ylabel()

plt.show()

5.12

If the input is a number:

$a=10$

1° if `type(a) == int` or `type(a) == float`:

`print("Yes")`

2° if `isinstance(a, int)` or `isinstance(a, float)`:

`print("Yes")`

pandas:

`import pandas as pd`

绘制 DataFrame = `data = { "name": ["Bob", "Alice", "John"],
"city": ["Lyon", "Paris", "Harbin"],
"age": [19, 18, 17] }` [a, b, c]
`tbl = pd.DataFrame(data=data, index=^)`

`tbl` is an object of Class DataFrame)
index-col=0

`tbl.shape` $\Rightarrow (3, 3)$

`tbl.dtypes` $\Rightarrow$

	name	city	age
object			
object			
int64			
dtype: object			

 $\Rightarrow$ object in pandas is the same as string in python

`tbl.head(1)` $\Rightarrow$

	name	city	age
a	Bob	Lyon	19

 the first row

`tbl.tail(1)` $\Rightarrow$

	name	city	age
c	John	Harbin	17

 the last row

Access col: `tbl["city"]` $\Rightarrow$

	city	age
a	Lyon	19
b	Paris	18
c	Harbin	17

Name: city, dtype: object

`tbl[["city", "age"]]` $\Rightarrow$

	city	age
a	Lyon	19
b	Paris	18
c	Harbin	17

Name: city age, dtype: object

Access row: `tbl.loc["b"]` $\Rightarrow$

	city	age
b	Paris	18

Name: b, dtype: object

`tbl.loc[["b", "c"]]` $\Rightarrow$

	city	age
b	Paris	18
c	Harbin	17

Access rows and cols: `tbl.loc[["b", "c"], ["name", "city"]]` $\Rightarrow$

	name	city
b	Alice	Paris
c	John	Harbin

`tbl.iloc[[1, 2], [0, 1]]` $\Rightarrow$

`tbl.iloc[1:3, 0:2]`

Access a single value: `tbl.iloc[0, 1]` $\Rightarrow$ 'Lyon'

Mask: `mask = [False, True, True]`

`tbl.loc[mask]` $\Rightarrow$

	name	city	age
b	Alice	Paris	18
c	John	Harbin	17

`(age > 17) mask = tbl["age"] > 17` $\Rightarrow$

	name	city	age
b	Alice	Paris	18
c	John	Harbin	17

`tbl.loc[mask]` $\Rightarrow$

`(city == "Paris") mask = tbl["age"] == "Paris"` $\Rightarrow$

	name	city	age
b	Alice	Paris	18

NaN: `data = {"name": ["Bob", "Alice", "John"],`
`"city": ["Lyon", "Paris", "Harbin"],`
`"age": [19, 18, np.nan]}`

`tbl = pd.DataFrame(data=data)`

$\Rightarrow$

	name	city	age
0	Bob	Lyon	19.0
1	Alice	Paris	18.0
2	John	Harbin	NaN

`tbl`

select the rows where the age is missing / NaN.

mask = tbl["age"].isna() / .isnull()

df[mask] $\Rightarrow$

name	city	age
John	Harbin	NaN

... not missing

mask = ~tbl["age"].isna() / .isnull()

mask: () & ()

分组:

df.groupby()

计数:

df[].value_counts()

df.groupby().size()

排列:

df.sort_values([] ascending=False / True)

Replace: 'yes' $\Rightarrow$ True / 'no' $\Rightarrow$ False

1° mask = df[] == "yes"

df.loc[mask,] = True

2° map

df[] = df[].map({ "yes": True, "no": False })

3° df[] = df[].apply(lambda v: True if v == "yes" else False)

4° def _ =

.apply(_)

5.13

Add cols to an existing DataFrame

```
import pandas as pd
df = pd.DataFrame(data = { "Name": ["A", "B"], "Age": [19, 18] })
```

```
df["City"] = ["Lyon", "Paris"]
```

⇒

	Name	Age	City
0	A	19	Lyon
1	B	18	Paris

```
df.insert(2, "Gender", ["M", "F"])
```

⇒

	Name	Age	Gender	City
0	A	19	M	Lyon
1	B	18	F	Paris

(where) index col name values

Remove cols

```
df.drop(columns = ["Gender", "City"])
```

⇒

	Name	Age
0	A	19
1	B	18

不要原来的 df

1° re-assign 1° `df = df.drop(columns = ["Gender", "City"])`

2° `df.drop(columns = ["Gender", "City"], inplace = True)`

Add Rows

```
df.loc[2] = ["C", 17]
```

⇒

	Name	Age
0	A	19
1	B	18
2	C	17

Remove a row

```
df = df.drop(index = [2])
```

⇒

	Name	Age
0	A	19
1	B	18

Add multiple rows (have the same columns)

df1	Name	Age	df2	Name	Age	df3	Name	Cray
0	A	19	0	C	19	0	A	Lyon
1	B	18	1	D	18	1	B	Paris

`pd.concat([df1, df2])` ⇒

	Name	Age
0	A	19
1	B	18
0	C	19
1	D	18

Add new cols

`df1.join(df3["Cray"])` ⇒

	Name	Age	Cray
0	A	19	Lyon
1	B	18	Paris

`df4:`

	Name	Cray
0	B	Lyon
1	D	Paris

`df1.merge(df4, on="Name")` ⇒

	Name	Age	Cray
0	B	18	Lyon

统计摘要

`df.dtypes`

`df.describe()` 生成数值型列 (numerical cols)
非缺失型数据 统计量
(count, mean, std, min, 25%, 50%, 75%, max)

`df.describe(include="o")` 非数值型列 (categorical cols)
唯一值数量 频率最高的值
(count, unique, top, freq)

`df.info()`

`df.columns` ⇒ Index([], dtype='')

`df.index` ⇒ Index([], dtype='')

5.15

lambda: inputs: return value

multiply = lambda x: x * 100

multiply(5) $\Rightarrow$ 500

use replace when you want to change only one value at a time
df[] = df[].replace(,) old value, new value

pivot table

df.pivot_table(index=" ", columns=" ", values=" ", ^{function}aggfunc=" ")

Find the most common type-combination of the dataset

df.groupby([,]).size().idxmax()

返回最大值所在索引

to convert a Series back to a DataFrame you can use
df.groupby([,]).size().reset_index(name=" ") ^{reset index}

rename:

cols_map = { : , : , : , : } old: new

df.rename(columns=cols_map)

check for missing values: `df.isnull().sum()`

Fill (impute) missing values in numerical cols:

with the average: `sgm_avg = df[ ].mean()`

`df[ ] = df[ ].fillna(sgm_avg)`

with the mode: `most_freq = df[ ].mode`

`df[ ] = df[ ].fillna(most_freq)`

`df.dropna()` Remove all the rows with at least missing values

`df[ ] = df[ ].astype(int)` change col data dtype

`nums = [1, 2, 3, 7, 8, 19, 31, 32, 33]`

`pd.cut(nums, bins=3, labels=["low", "mid", "high"])`

⇒ `['low', 'low', 'low', 'low', 'low', 'mid', 'high', 'high', 'high']`

creates equal-ranged groups

`pd.qcut(nums, q=3, labels=["low", "mid", "high"])`

⇒ `['low', 'low', 'low', 'mid', 'mid', 'mid', 'high', 'high', 'high']`

creates equal-sized groups

Convert categorical values to numbers

`names = ['A', 'B', 'A', 'C', 'C']`

⇒ `array([0, 1, 0, 2, 2])`

from `sklearn.preprocessing` import `LabelEncoder`

`encoder = LabelEncoder()` first create a object of encoder

`encoder.fit_transform(names)` encoder values to number

`pd.get_dummies(names)` $\Rightarrow$

	A	B	C
0	True	False	False
1	False	True	False
2	True	False	False
3	False	False	True
4	False	False	True

Loop over cols in pandas

for `c` in `df.columns`:

`print(c, df[c].dtype)`

Loop over rows in pandas

for `idx, row_values` in `df.iterrows()`:

`print(idx, row_values)`

zip:

`a = [1, 2, 3]`

`b = ["A", "B", "C"]`

for `v1, v2` in `zip(a, b)`:

`print(v1, v2)`

$\Rightarrow$

1	A
2	B
3	C

哈尔滨工业大学
薪火笔记社

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
box: pandas: df_low[ ].plot(kind="box")  
matplotlib: x=df_low[ ].values  
plt.boxplot(x)
```

```
seaborn = sns.boxplot(data=df_low, x=" ")
```

```
hist: pandas: df_low[ ].plot(kind='hist') ylabel='Frequency'  
matplotlib: x=df_low[ ].values  
plt.hist(x) ylabel='Count'
```

```
x=df_low[ ].value_counts().index  
y=df_low[ ].value_counts().values  
plt.hist(x)
```

```
seaborn = sns.histplot(data=df_low, x=" ") #223  
sns.displot(data=df_low, x=" ", kde=True) #223  
"energy" xlabel='energy'
```

```
bar: pandas: df_low[ ].value_counts().plot(kind="bar")  
matplotlib: x=df_low[ ].value_counts().index  
y=df_low[ ].value_counts().values  
plt.bar(x)
```

```
seaborn = sns.barplot(df_low[ ].value_counts())
```

Scatter: pandas: `df.plot(kind="scatter", x=" ", y=" ")`

matplotlib: `x = df[ ]`

`y = df[ ]`

`plt.scatter(x, y)`

Seaborn: `sns.scatterplot(data=df, x=" ", y=" ")`

matplotlib and seaborn: `plt.figure(figsize=(10, 4))`

`sns.boxplot(data=df_low, x=" ", y=" ")`

`plt.xticks(rotation=45)`

哈尔滨工业大学

薪火笔记社

`plt.figure(figsize=(10, 4))`

`sns.boxplot(data=df_low, x=" ", y=" "; hue=" ")`

Summary:

Create Test Objects

Operator	Description
<code>pd.DataFrame(np.random.rand(20,5))</code>	5 columns and 20 rows of random floats ✓
<code>pd.Series(my_list)</code>	Create a series from an iterable my_list
<code>df.index = pd.date_range('1900/1/30', period = df.shape[0])</code>	Add a date index

Viewing/Inspecting Data

Operator	Description
<code>df.head(n)</code>	First n rows of the DataFrame
<code>df.tail(n)</code>	Last n rows of the Data Frame
<code>df.shape</code>	Number of rows and columns
<code>df.info()</code>	Index, Datatype and Memory information ✓
<code>df.describe()</code>	Summary statistics for numerical columns
<code>S.value_counts(dropna=False)</code>	View unique values and counts columns
<code>df.apply(pd.Series.value_counts)</code>	Unique values and counts for all ✓

Selection

Operator	Description
<code>df[col]</code>	Returns column with label col as Series
<code>df[[col1, col2]]</code>	Returns columns as a new DataFrame
<code>s.iloc[2]</code>	Selection by position
<code>s.loc['index_one']</code>	Selection by Index
<code>df.iloc[0,:]</code>	First row
<code>df.iloc[0,0]</code>	First element of first column

Data Cleaning

Operator	Description
<code>df.columns = ['a', 'b', 'c']</code>	Rename columns
<code>pd.isnull()</code>	Checks for null values, Returns Boolean Array
<code>pd.notnull()</code>	Opposite of <code>pd.isnull()</code>
<code>df.dropna()</code>	Drop all rows that contain null values
<code>df.dropna(axis=1)</code>	Drop all columns that contain null values
<code>df.dropna(axis=1, thresh=n)</code>	Drop all rows have less than n non null values
<code>df.fillna(x)</code>	Replace all null values with x
<code>s.fillna(s.mean())</code>	Replace all null values with the mean
<code>s.astype(float)</code>	Convert the data type of the series to float
<code>s.replace([2,3], ['two', 'three'])</code>	Replace all values equal to 1 with 'one' all 2 with 'two' and 3 with 'three'
<code>df.rename(columns = {old_name: 'new_name'})</code>	Selective renaming

`df.rename(columns=lambda x: x+1)`
`df.set_index('column-one')`
`df.rename(index=lambda x: x+1)`

Mass renaming of columns
Change the index
Mass renaming of index

Filter, Sort, and Groupby

Operator

`df[df[col]>0.6]`
`df[(df[col]>0.6)&(df[col]<0.8)]`
`df.sort_values(col)`
`df.sort_values(cols, ascending=False)`
`df.sort_values([col1, col2], ascending=[True, False])`
`df.groupby(col)`
`df.groupby([col1, col2])`
`df.groupby(col)[col2]`
`df.pivot(index=col,
values=[col2, col3], aggfunc='mean')`
`df.groupby(col).agg(np.mean)`
`df.apply(np.mean)`
`np.apply(np.mean, axis=1)`

Description

Rows where the column col is greater than 0.6
Rows where $0.8 > col > 0.6$
Sort values by col1 in ascending order
Sort values by col2 in ascending order
Sort values by col1 in ascending order
Sort values by col1 in ascending order
Returns a grouping object for values from ^{one column} multiple columns
Returns grouping object for values from ^{multiple columns} _{by the values in col1}
Returns the mean of the values in col2, grouped
Create a pivot table that groups by col1 and calculates
the mean of col2 and col3
_{col1 group}
Find the average across all columns for every unique
Apply the function `np.mean()` across each column
Apply the function `np.mean()` across each row

Join / Combine

Operator	Description
<code>df1.append(df2)</code>	Add the rows in <code>df1</code> to the end of <code>df2</code>
<code>pd.concat([df1, df2], axis=1)</code>	Add the columns in <code>df1</code> to the end of <code>df2</code>
<code>df1.join(df2, on=col1, how='inner')</code>	SQL-style join the columns in <code>df1</code> with the columns on <code>df2</code> where the rows for <code>col</code> have identical values

Statistics

Operator	Description
<code>df.describe()</code>	Summary statistics for numerical columns
<code>df.mean()</code>	Returns the mean of all columns
<code>df.corr()</code>	Returns the correlation between columns in a DataFrame
<code>df.count()</code>	Returns the number of non-null values in each ^{Column} DataFrame
<code>df.max()</code>	Returns the highest value in each column
<code>df.min()</code>	Returns the lowest value in each column
<code>df.median()</code>	Returns the median of each column
<code>df.std()</code>	Returns the standard deviation of each column

Importing Data

Operator	Description
<code>pd.read_csv(filename)</code>	From a CSV file
<code>pd.read_table(filename)</code>	From a delimited text file (like TSV)
<code>pd.read_excel(filename)</code>	From an Excel file
<code>pd.read_sql(query, connection-object)</code>	Read from a SQL table/database
<code>pd.read_json(json_string)</code>	Read from a JSON formatted string, URL or file.
<code>pd.read_html(url)</code>	Parses an HTML URL, string or file and extracts tables ^{to a list of data frames}
<code>pd.read_clipboard()</code>	Takes the contents of your clipboard and passes it ^{to read-table()}
<code>pd.DataFrame()</code>	From a dict, keys for column names, values for data ^{as lists}

Exporting Data

Operator	Description
<code>df.to_csv(filename)</code>	Write to a CSV file
<code>df.to_excel(filename)</code>	Write to an Excel file
<code>df.to_sql(table-name, connection-object)</code>	Write to a SQL table
<code>df.to_json(filename)</code>	Write to a file in JSON format

哈尔滨工业大学
薪火笔记社